

郝政伟答辩闭环：组长总述、演示与全场兜底

1. 先记住自己的定位

你不是要把每个模块都讲到最深，而是要完成四件事：

1. 用 3 分钟把“做成了什么、怎么设计、怎么证明”讲清楚。
2. 现场演示 1 个成功样例和 1 个错误样例。
3. 老师点到某个模块时，立刻把话题转给对应同学。
4. 如果组员卡住，你能用 1 到 2 句话把问题兜回主线。

一句总括可以直接记：

我们做的是 Pascal-S 到 C 的源到源编译器，已经形成完整流水线，并且不只平台 95/95，还补充了 `record`、`repeat-until`、`caret diagnostics`、`panic-mode` 恢复和测试闭环。

2. 现场先打开哪些文件

你现场最该打开的不是一堆源码，而是：

1. [docs/presentation/final_defense.pdf](#)
2. 终端或 PowerShell 窗口
3. 如果老师追问主控流程，再补开：
 - [src/main.cpp](#)
 - [src/semantic_analyzer.cpp](#)
 - [src/codegen.cpp](#)

你自己最该记住的代码定位只有这 3 处：

- [src/main.cpp:1](#)
 - [src/semantic_analyzer.cpp:478](#)
 - [src/codegen.cpp:1](#)
-

3. 组长 3 分钟总述怎么讲

不要每页平均讲，按这个顺序讲。

第 1 页：封面，10 秒

我们组完成的是一个 Pascal-S 到 C 的源到源编译器，核心流程包括词法分析、语法分析、语义检查和 C11 代码生成。

第 2 页：成果总览，25 秒

从结果上看，Educoder 平台 95/95 全通过；另外补充实现了 `record`、`repeat-until`、`caret` `diagnostics`，并做了课程自建测试和严格 C11 检查。

第 3 页：整体架构，35 秒

整体架构是 `.pas -> lexer -> parser/AST -> semantic -> codegen -> .c`。

这里最关键的是 AST 作为统一中间表示：parser 只建树，semantic 负责合法性检查，codegen 在语义正确后生成 C。

第 4 页：核心设计，30 秒

这样拆分的原因是把语法和语义解耦。如果把符号表、类型判断和扩展功能都塞进 parser，维护会很困难。现在前端、中端、后端职责更清楚。

第 5 页：语义分析示例，30 秒

比如 `node.missing := 1`，语义阶段会先查 `node` 的声明，再确认它是 `record`，最后检查字段表，发现 `missing` 不存在后直接报错。

第 6 页：代码生成映射，25 秒

代码生成的核心不是简单打印字符串，而是做 Pascal-S 到 C 的语义映射，例如 `record -> struct`，`var` 参数 -> 指针参数，函数名赋值 -> `_retval`，数组非零下界 -> 下标偏移。

第 7 页：错误诊断与恢复，20 秒

错误处理上，我们实现了 `caret diagnostics`，会显示源码原文和 `^~~~` 定位箭头；语法阶段还支持有限的 panic-mode 恢复，可以越过第一处错误继续报告后续独立错误。

第 8 页：测试验证，20 秒

验证方面不只平台 95/95。我们还有课程自建测试 16/16、严格 C11 检查 9/9，说明项目不是只靠样例碰运气。

第 9 页：分工与总结，15 秒

最后我们按模块分工，由组长统一集成。整体上完成了 Pascal-S 到 C 的完整翻译流程，并在语言扩展、错误诊断和测试闭环上形成了较明确的亮点。

4. 现场演示怎么做

只演示两条：

1. 一个成功样例
2. 一个错误样例

推荐就是这两条。

成功样例: record_nested_access

```
. \build-mingw2\pascc.exe -i . \course_tests\success\record_nested_access.pas
gcc . \course_tests\success\record_nested_access.c -o
. \course_tests\success\record_nested_access.exe
. \course_tests\success\record_nested_access.exe
```

预期输出:

7

你讲的时候这样说:

这里先把 Pascal-S 编译成 C, 再用 gcc 编译并运行。输出结果是 7, 说明生成代码可继续编译和执行。

错误样例: semantic_record_unknown_field

```
. \build-mingw2\pascc.exe . \course_tests\errors\semantic_record_unknown_field.pas
```

预期现象:

- 输出 Semantic Error
- 显示源码行
- 显示 ^~~~~~

你讲的时候这样说:

这里展示的是 record 字段不存在时的语义错误。除了错误类型和行列号, 还会打印源码行和 ^~~~~ 箭头定位。

5. 你怎么把问题转给对应同学

词法分析

这部分请张睿渤补充, 他主要讲 token 规则、注释处理和行列号记录, 对应文件是 [src/lexer.l](#)。

语法分析

这部分请周柄名补充, 他主要讲 Bison 文法、AST 构建和 panic-mode 恢复, 对应文件是 [src/parser.y](#)。

AST 与公共数据结构

这部分请蔡禹煊补充, 他主要讲 AST 节点分层和访问者模式, 对应文件是 [src/ast.h](#)。

符号表与语义分析

这部分请李良顺补充，他主要讲作用域栈符号表、声明绑定和类型检查，对应文件是 [src/symbol_table.h](#) 和 [src/semantic_analyzer.cpp](#)。

类型系统与错误诊断

这部分请孔力帆补充，他主要讲统一类型查询和 `caret_diagnostics`，对应文件是 [src/type_resolver.cpp](#) 和 [src/error.cpp](#)。

测试、运行支撑与使用说明

这部分请吴锡华补充，他主要讲自建测试、严格 C11 检查和使用说明，对应文件是 [course_tests/run_course_tests.ps1](#)。

6. 如果组员卡住，你怎么兜底

词法卡住

这一层主要是把字符流稳定切成 token，并把位置信息保留下来，供 parser 和错误诊断继续使用。

语法卡住

这一层输出的是 AST，不做类型判断；真正的合法性检查放到后面的语义分析阶段。

AST 卡住

AST 是统一中间表示，parser 建树，semantic 检查树，codegen 生成树对应的目标代码。

符号表/语义卡住

符号表解决的是名字和作用域，类型查询回答“表达式是什么类型”，语义分析再据此判断是否合法。

类型系统/错误诊断卡住

`TypeResolver` 和 `ErrorHandler` 都是公共支撑层：前者统一类型查询，后者统一输出 `caret_diagnostics`。

测试卡住

我们的测试不是只跑平台样例，还包括成功样例、错误样例和严格 C11 检查三层验证。

7. 老师最可能直接问你的问题

问题 A：你们的项目亮点到底是什么？

标准答法：

亮点主要有四个：

第一，编译流程做成了较清晰的前端、中端、后端拆分；

第二，扩展支持了 `record` 和 `repeat-until`；

第三，错误诊断从简单行号升级到了 `caret diagnostics`，并在语法阶段支持有限的 panic-mode 恢复；

第四，除了平台 95/95，还补了自建测试和严格 C11 检查。

问题 B：为什么你们要把语义分析独立出来？

标准答法：

因为语法正确不代表语义正确。变量未声明、参数不匹配、字段不存在这些都不是 parser 该解决的，所以我们把声明绑定、作用域管理和类型检查独立到了 `SemanticAnalyzer` 这一层。

问题 C：代码生成输入是什么？

标准答法：

输入是已经通过语义检查的 AST，以及符号和类型信息。代码生成不会重新做一遍语义判定，而是在语义正确的前提下做 Pascal-S 到 C 的映射。

问题 D：错误处理与恢复策略是什么？

标准答法：

词法、语法、语义阶段都会走统一的错误输出；语法阶段实现了有限的 panic-mode 恢复，可以在同步点继续分析；语义和代码生成阶段则采用停止式处理，优先保证错误定位准确和输出结果可靠。

8. 最短保底版：45 秒回答

如果你一下子被打断或者脑子短路，只背这段也能站住：

我作为组长主要负责整体架构、模块联调、测试验收和答辩整合。项目做的是 Pascal-S 到 C 的源到源编译器，当前已经完成词法分析、语法分析、语义分析和 C11 代码生成，平台测试 95/95 全通过。另外还扩展实现了 `record`、`repeat-until`、`caret diagnostics` 和语法阶段的 panic-mode 恢复，并通过课程自建测试和严格 C11 检查做了补充验证。

9. 你最容易说错的 6 个点

1. 不要说“我们实现了机器级重定位”。

正确说法：输出是 C 源码，后续编译和链接由 C 编译器完成。

2. 不要说“错误恢复都在 `error.cpp` 里做”。

正确说法：恢复在 parser 的 `error` 规则里，`ErrorHandler` 负责统一输出和去重。

3. 不要把 `record` 和 `struct` 混着叫。

正确说法：Pascal-S 里是 `record`，生成 C 时映射成 `struct`。

4. 不要把 `TypeResolver` 说成“又做了一遍语义分析”。

正确说法：它是统一的类型查询入口。

5. 不要把平台通过率当成唯一亮点。

更好的说法是：平台结果 + 扩展功能 + 错误诊断 + 测试闭环。

6. 不要现场演示太多。

两条命令足够：一个成功、一个失败。

10. 郝政伟只需要记住的文件定位

最推荐只记下面这些：

- [docs/presentation/final_defense.pdf](#)
- [src/main.cpp:1](#)
- [src/semantic_analyzer.cpp:478](#)
- [src/codegen.cpp:1](#)
- [course_tests/run_course_tests.ps1:27](#)
- [course_tests/run_course_tests.ps1:74](#)

这几处足够支撑你总述、演示和兜底。